

Natural Language Processing

2. Words and tokens

Paolo Torroni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Regular Expressions
- Tokenization
- Normalization, Lemmatization and Stemming
- Datasets and Tools
- Edit Distance
- Spelling Correction

Regular Expression

Regular expressions

Men are all alike.

IN WHAT WAY

They're always bugging us about something or other.

CAN YOU THINK OF A SPECIFIC EXAMPLE

Well, my boyfriend made me come here.

YOUR BOYFRIEND MADE YOU COME HERE

He says I'm depressed much of the time.

I AM SORRY TO HEAR YOU ARE DEPRESSED

`s/(.*)/\U\1/`

`s/(.*) MY (.*)/YOUR \2/`

`s/(.*) ME (.*)/\1 YOU \2/`

`s/.* I'M (DEPRESSED|SAD) .*/I AM SORRY TO HEAR YOU ARE \1/`

`s/.* I AM (DEPRESSED|SAD) .*/WHY DO YOU THINK YOU ARE \1/`

`s/.* ALL .*/IN WHAT WAY/`

`s/.* ALWAYS .*/CAN YOU THINK OF A SPECIFIC EXAMPLE/`

Why regular expressions

- To describe text patterns
- To extract information from text
 - Stuffed Animals & Toys under \$10
- To convert text into a convenient/standard form

Text normalization

- **Tokenization**
 - I'm an #Englishman in New York ;)
- **Lemmatization and stemming**
 - sang, sung, sings, sing
- **Sentence segmentation**

Regular expressions

- A formal language for specifying text strings
- How can we search for any of these?
 - woodchuck
 - woodchucks
 - Woodchuck
 - Woodchucks



Disjunctions

- Letters inside square brackets `[]`

RE	Match	Example Patterns
<code>/[wW]oodchuck/</code>	Woodchuck or woodchuck	“ <u>W</u> oodchuck”
<code>/[abc]/</code>	‘a’, ‘b’, or ‘c’	“In uomini, in soldati”
<code>/[1234567890]/</code>	any digit	“plenty of <u>7</u> to 5”

Figure 2.2 The use of the brackets `[]` to specify a disjunction of characters.

- Ranges `[A-Z]`

RE	Match	Example Patterns Matched
<code>/[A-Z]/</code>	an upper case letter	“we should call it ‘ <u>D</u> renched Blossoms’ ”
<code>/[a-z]/</code>	a lower case letter	“ <u>m</u> y beans were impatient to be hoed!”
<code>/[0-9]/</code>	a single digit	“Chapter <u>1</u> : Down the Rabbit Hole”

Figure 2.3 The use of the brackets `[]` plus the dash `-` to specify a range.

Negation in Disjunction

- Negations `[^Ss]`
 - Caret means negation only when first in `[]`

RE	Match (single characters)	Example Patterns Matched
<code>/[^A-Z]/</code>	not an upper case letter	“Oy <u>f</u> n pripetchik”
<code>/[^Ss]/</code>	neither ‘S’ nor ‘s’	“ <u>I</u> have no exquisite reason for’t”
<code>/[^.]/</code>	not a period	“ <u>o</u> ur resident Djinn”
<code>/[e^]/</code>	either ‘e’ or ‘^’	“look up <u>^</u> now”
<code>/a^b/</code>	the pattern ‘a^b’	“look up <u>a^</u> b now”

Figure 2.4 The caret `^` for negation or just to mean `^`. See below re: the backslash for escaping the period.

More Disjunction

- Woodchuck is another name for groundhog!
- The pipe `|` for disjunction
 - `groundhog|woodchuck`
 - `yours|mine`
 - `a|b|c`
 - `[gG]roundhog| [wW]oodchuck`



Optional Elements and Wildcards: ?, *, +, .

- ? Optional previous character
- * Zero or more instances of the previous character (**Kleene ***)
- + One or more instances of the previous character (**Kleene +**)
- . Any character

RE	Match	Example Patterns Matched
/woodchucks?/	woodchuck or woodchucks	" <u>woodchuck</u> "
/colou?r/	color or colour	" <u>color</u> "

Figure 2.5 The question mark ? marks optionality of the previous expression.

RE	Match	Example Matches
/beg.n/	any character between <i>beg</i> and <i>n</i>	<u>begin</u> , <u>beg'n</u> , <u>begun</u>

Figure 2.6 The use of the period . to specify any character.

Anchors

- `^` start of a line (caret): `/^The/`
- `$` end of a line: `/^The dog\.$/`
- b** Word boundary: `/\bthe\b/`
- B** Word non-boundary: `/\Bthe\B/`

More regex Operators

- Common sets of characters

RE	Expansion	Match	First Matches
\d	[0-9]	any digit	Party_of_5
\D	[^0-9]	any non-digit	Blue_moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!
\s	[\r\t\n\f]	whitespace (space, tab)	
\S	[^\s]	Non-whitespace	in_Concord

Figure 2.7 Aliases for common sets of characters.

More regex Operators

- Common sets of characters
- Counting

RE	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	exactly zero or one occurrence of the previous char or expression
{ <i>n</i> }	<i>n</i> occurrences of the previous char or expression
{ <i>n</i> , <i>m</i> }	from <i>n</i> to <i>m</i> occurrences of the previous char or expression
{ <i>n</i> ,}	at least <i>n</i> occurrences of the previous char or expression
{, <i>m</i> }	up to <i>m</i> occurrences of the previous char or expression

Figure 2.8 Regular expression operators for counting.

More regex Operators

- Common sets of characters
- Counting
- Escape symbol (backslash)

RE	Match	First Patterns Matched
*	an asterisk “*”	“K*_A*P*L*A*N”
\.	a period “.”	“Dr. Livingston, I presume”
\?	a question mark	“Why don’t they come and lend a hand?”
\n	a newline	
\t	a tab	

Figure 2.9 Some characters that need to be backslashed.

Substitution

- Importantly, regular expressions are used in **substitutions**

Men are all alike.
IN WHAT WAY
They're always bugging us about something or other.
CAN YOU THINK OF A SPECIFIC EXAMPLE
Well, my boyfriend made me come here.
YOUR BOYFRIEND MADE YOU COME HERE
He says I'm depressed much of the time.
I AM SORRY TO HEAR YOU ARE DEPRESSED

```
s/(.*)/\U\1/  
s/(.*) MY (.*)/YOUR \2/  
s/(.*) ME (.*)/\1 YOU \2/  
s/.* I'M (DEPRESSED|SAD) .*/I AM SORRY TO HEAR YOU ARE \1/  
s/.* I AM (DEPRESSED|SAD) .*/WHY DO YOU THINK YOU ARE \1/  
s/.* ALL .*/IN WHAT WAY/  
s/.* ALWAYS .*/CAN YOU THINK OF A SPECIFIC EXAMPLE/
```


Capture Groups

- Substitutions make use of **capture groups** to refer to specific subparts of the string matching a pattern

```
/the (.*?)er they were, the \1er they will be/
```

```
/the (.*?)er they (.*), the \1er we \2/
```

- to use parentheses without capturing: `(?: ...)` (non-capturing group)

Online regex tester and debugger: regex101.com

Tools

- Regexes are **built-in in most programming languages**, as well as in word processors and shell utilities such as `sed` and `awk`
- `sed` (a *stream editor* that comes with Linux) modifies the input according to patterns expressed in some form of regular expression
`sed -E 's/I (.*) my (.*)/YOU \1 YOUR \2/g'`
- in Python, you can use `re`

For fun: Eliza in Python

```
import re

input_text = ['Men are all alike.',
'They\'re always bugging us about something or other.',
'Well, my boyfriend made me come here.',
'He says I\'m depressed much of the time.']

def reply(line):
    newline = line.upper()
    newline = re.sub(r'.* ALL .*', 'IN WHAT WAY', newline)
    newline = re.sub(r'.* ALWAYS .*', 'CAN YOU THINK OF A SPECIFIC EXAMPLE', newline)
    newline = re.sub(r'(.*)\bMY\b(.*)\.', r'YOUR\2', newline)
    newline = re.sub(r'\bME\b', r'YOU', newline)
    newline = re.sub(r'.*\b(?:I\'M|I AM) (DEPRESSED|SAD)\b.*',
        r'I\'M SORRY TO HEAR THAT YOU ARE \1', newline)

    print(line)
    print(newline)

for line in input_text:
    reply(line)
```

```
Men are all alike.
IN WHAT WAY
They're always bugging us about something or other.
CAN YOU THINK OF A SPECIFIC EXAMPLE
Well, my boyfriend made me come here.
YOUR BOYFRIEND MADE YOU COME HERE
He says I'm depressed much of the time.
I\'M SORRY TO HEAR THAT YOU ARE DEPRESSED
```

Tokenization

Text Normalization

- Every NLP task needs to do text normalization:
 - 1 Segmenting/tokenizing words in running text
 - 2 Normalizing word formats
 - 3 Segmenting sentences in running text

How Many Words?

- *I do uh main- mainly business data processing*
 - Two kinds of **disfluencies**: fragments and filled pauses
 - Are these words?
- *Seuss's **cat** in the hat is different from other **cats**!*
 - **Lemma**: same stem, part of speech, rough word sense
 - **cat** and **cats** = same lemma
 - **Wordform**: the full inflected surface form
 - **cat** and **cats** = different wordforms

How Many Words?

- *they lay back on the San Francisco grass and looked at the stars and their*
 - **Type**: an element of the vocabulary
 - **Token**: an instance of that type in running text
 - How many tokens/types?

How Many Words?

- **N** = number of tokens
- **V** = vocabulary = set of types
 - $|V|$ is the size of the vocabulary
 - Herdan's Law (1960): $|V| = kN^\beta$

Corpus	Tokens = N	Types = $ V $
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13 million

Figure 2.10 Rough numbers of types and tokens for some English language corpora. The largest, the Google N-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

In these corpora, β ranges from .67 to .75

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < sh.txt
```

```
THE  
SONNETS  
by  
William  
Shakespeare  
From  
fairest  
creatures  
We  
...
```

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < sh.txt | sort | uniq -c
```

```
1945 A
72 AARON
19 ABBESS
25 Aaron
6 Abate
1 Abates
5 Abbess
6 Abbey
```

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c
```

```
14725 a
 97 aaron
  1 abaissiez
10 abandon
  2 abandoned
  2 abase
  1 abash
14 abate
  3 abated
  3 abatement
...
```

Simple Tokenization in UNIX

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c | sort -n -r
```

```
27378 the
26084 and
22538 i
19771 to
17481 of
14725 a
13826 you
12489 my
11318 that
11112 in
...
```

- The most frequent words turn out to be short **function words** like articles, pronouns, prepositions

Issues in Tokenization

- Finland's capital
- what're, I'm, isn't (**clitic** contractions)
- Hewlett-Packard
- state-of-the-art
- Lowercase
- San Francisco, rock 'n' roll (multiword expression dictionary, **NER?**)
- m.p.h., PhD.
- \$45.50
- 555,500.00
- 01/02/06
- <http://www.unibo.it>
- #cool

Standard Tokenization

- There are standards for tokenization
- For speed, tokenization usually made by deterministic algorithms based on regex compiled into very efficient finite state automata

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...     | \w+?:(-\w+)*      # words with optional internal hyphens
...     | \$?\d+(?:(\.\d+)?)? # currency, percentages, e.g. $12.40, 82%
...     | \.\.\.           # ellipsis
...     | [][.,;'"'?:()_-' ] # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Figure 2.12 A Python trace of regular expression tokenization in the NLTK Python-based natural language processing toolkit (Bird et al., 2009), commented for readability; the (?x) verbose flag tells Python to strip comments and whitespace. Figure from Chapter 3 of Bird et al. (2009).

Tokenization: Language Issues

French

- **L'ensemble**: one token or two?
- Want **l'ensemble** to match with **un ensemble**

German noun compounds are not segmented

- **Lebensversicherungsgesellschaftsangestellter**
- Life insurance company employee
- German information retrieval needs **compound splitter**

Chinese words composed of characters (**hanzi**) generally representing a single unit of meaning (**morpheme**)

- deciding what counts as a word is complex
- for most Chinese NLP tasks better taking characters as input

Data-Driven Tokenization Algorithms

- Alternative to rule-based tokenization
- Especially effective in dealing with *unknown words*
- Tokens could be smaller than words (**subwords**)
- Subwords could be arbitrary substrings, or meaning-bearing units like the morphemes *un-*, *-est* or *-er*
- Idea: use a large corpus to learn what a “token” should be
- Two-fold tokenization scheme:
 - **token learner**, induces a vocabulary from a corpus
 - **token segmenter**, does the segmentation using the learned vocabulary
- Commonest algorithms: **Byte-Pair Encoding (BPE)**, **unigram language modeling**, **WordPiece**, **tiktoken**

Neural Machine Translation of Rare Words with Subword Units, R Sennrich et al, ACL 2016

Byte Pair Encoding: Token Learner

- Originally a compression algorithm
 - most frequent *byte* pair $\rightarrow$ a new *byte*
- Use for word segmentation: replace “byte” with “character ngram”
 - Start with a unigram vocabulary of all (Unicode) characters in data
 - most frequent ngram pair $\rightarrow$ a new ngram

Vocabulary: {_,l,o,w,e,r,n,w,s,t,i,d}

Dictionary:

5 l o w _

2 l o w e r _

6 n e w e s t _

3 w i d e s t _

Byte Pair Encoding: Token Learner

- Originally a compression algorithm
 - most frequent *byte* pair $\rightarrow$ a new *byte*
- Use for word segmentation: replace “byte” with “character ngram”
 - Start with a unigram vocabulary of all (Unicode) characters in data
 - most frequent ngram pair $\rightarrow$ a new ngram

Vocabulary: {_,l,o,w,e,r,n,w,s,t,i,d,**es**}

Dictionary:

5 l o w _

2 l o w e r _

6 n e w **es** t _

3 w i d **es** t _

Byte Pair Encoding: Token Learner

- Originally a compression algorithm
 - most frequent *byte pair* → a new *byte*
- Use for word segmentation: replace “byte” with “character ngram”
 - Start with a unigram vocabulary of all (Unicode) characters in data
 - most frequent ngram pair → a new ngram

Vocabulary: {_,l,o,w,e,r,n,w,s,t,i,d,es,**est**}

Dictionary:

5 l o w _

2 l o w e r _

6 n e w **est** _

3 w i d **est** _

Byte Pair Encoding: Token Learner

- Originally a compression algorithm
 - most frequent *byte* pair $\rightarrow$ a new *byte*
- Use for word segmentation: replace “byte” with “character ngram”
 - Start with a unigram vocabulary of all (Unicode) characters in data
 - most frequent ngram pair $\rightarrow$ a new ngram

Vocabulary: {_,l,o,w,e,r,n,w,s,t,i,d,e,s,e,s,t,**est**_}

Dictionary:

5 l o w _
2 l o w e r _
6 n e w **est**_
3 w i d **est**_

Byte Pair Encoding: Token Learner

- Originally a compression algorithm
 - most frequent *byte* pair $\rightarrow$ a new *byte*
- Use for word segmentation: replace “byte” with “character ngram”
 - Start with a unigram vocabulary of all (Unicode) characters in data
 - most frequent ngram pair $\rightarrow$ a new ngram
- Segmentation only within words identified by some prior tokenizer
- **Automatically decides** vocab for system
 - Have a target vocabulary size and stop when you reach it

BPE Algorithm: Token Learner

```
function BYTE-PAIR ENCODING(strings  $C$ , number of merges  $k$ ) returns vocab  $V$   
  
   $V \leftarrow$  all unique characters in  $C$            # initial set of tokens is characters  
  for  $i = 1$  to  $k$  do                           # merge tokens til  $k$  times  
     $t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$   
     $t_{NEW} \leftarrow t_L + t_R$                    # make new token by concatenating  
     $V \leftarrow V + t_{NEW}$                          # update the vocabulary  
    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$  # and update the corpus  
  return  $V$ 
```

Figure 2.13 The token learner part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from [Bostrom and Durrett \(2020\)](#).

Byte pair encoding is suboptimal for language model pretraining, K Bostrom and G Durrett, EMNLP 2020

BPE Tokenization: Token Segmenter

- Once vocabulary is fixed and merge rules are defined, run token segmenter to tokenize input
- ① Segment input into single-character ngrams
- ② Parse sequence iteratively to identify possible merges
 - At each iteration, merge if merge rules applies
 - Otherwise, leave word split in multiple subwords
 - for example: **low est**_
 - If token not in vocabulary, [UNK]

For additional details and implementation, see [Byte-Pair Encoding tokenization](#) on [huggingface.co](#)

Other Tokenization Methods

- BPE used by GPT, GPT-2, RoBERTa, BART, DeBERTa, ...
- Variety of tokenization methods
- **WordPiece**
 - used by BERT, DistilBERT, MobileBERT and others
 - ## prefix to identify subwords
 - ranks possible merge rules based on a different score
- **Unigram** algorithm
 - used in SentencePiece (ALBERT, T5, mBART, Big Bird, XLNet)
 - starts from big vocab
 - remove symbols based on a loss function until desired size
- Visualize them! <https://tiktokenizer.vercel.app/>

For additional details and implementation, see [Byte-Pair Encoding tokenization](#) on huggingface.co

Tokenization in Multilingual Models

- The training data for large language models is vastly dominated by English text
- Multilingual BPE tokenizers tend to use most of the tokens for English, leaving fewer of them for other languages
- Result: worse segmentation, higher “fertility”, lower efficiency

In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·ginger,·and·nutmeg.

English: 14 words, 18 tokens

En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con·el·azúcar,·jengibre,·y·nuez·moscada.

Spanish: 16 words, 33 tokens

Normalization, Lemmatization and Stemming

Normalization

- Word **normalization** is the task of putting words/tokens in a standard format
 - E.g., Information Retrieval: we want to match **U.S.A.** and **USA**
 - Indexed text and query terms must have same form
- **Case folding** (e.g. mapping everything to lowercase) is another kind of normalization.
 - Are **US** and **us** the same?
 - Useful in IR and speech recognition
 - May be less so in sentiment analysis, information extraction and machine translation

Lemmatization

- Sometimes useful to have morphologically different words behave similarly
- E.g., looking for **restaurants** nearby vs **restaurant** nearby
- **Lemmatization** reduces inflections or variant forms to base form
 - am are, is → be
 - car, cars, car's, cars' → car
- **He is reading detective stories** → **He be read detective story**
- Accurate lemmatization requires complete **morphological parsing**
 - **stems**: the core meaning-bearing units
 - **affixes**: bits and pieces that adhere to stems
- Sometimes a simpler (cruder) method is enough: stemming

Stemming

- Naive version of morphological analysis
- Reduces terms to their stems
- Crude chopping of affixes
 - language dependent
 - e.g., automate(s), automatic, automation → automat
- **Porter Stemmer** is widely available and widely used
 - Cascading rewrite rules, such as:
 - ATIONAL → ATE
 - ING → ϵ
 - SSES → SS

Sentence Segmentation

- Most useful cues: punctuation
 - Question marks and exclamation points are relatively unambiguous
 - Periods are more ambiguous
 - E.g. Mr., Inc.
 - In the **Stanford CoreNLP** toolkit, sentence splitting is rule-based

Datasets and Tools

Playing with Words in Python: NLTK

- Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit
- Provides various corpora and tools including tokenizer, stemmers, lemmatizers

```
>>> import nltk
>>> nltk.download()
```

Collections	Corpora	Models	All Packages
Identifier	Name	Size	Status
all	All packages	n/a	not installed
all-corpora	All the corpora	n/a	not installed
book	Everything used in the NLTK Book	n/a	not installed

Download Refresh

Server Index: http://nltk.googlecode.com/svn/trunk/nltk_data/index.xml

Download Directory: C:\nltk_data

IMDB Movie Reviews Dataset

- [Large Movie Review Dataset](#) by Maas et al.
- A dataset for **binary sentiment classification** containing a set of 25,000 highly polar movie reviews for training, and 25,000 for testing.

 For fun: play with NLTK text normalization tools

AL Maas, RE Daly, PT Pham, D Huang, AY Ng, C Potts, [Learning Word Vectors for Sentiment Analysis](#), ACL 2011.

Your Turn: Use Regular Expressions to Tidy Up Text

```
# A) read dataset from file into memory
```

```
path = '/Users/paolo/datasets/aclImdb/movie_data/'
reviews_train = []
for line in open(path+'full_train.txt', 'r'):
    reviews_train.append(line.strip())
```

```
reviews_test = []
for line in open(path+'full_test.txt', 'r'):
    reviews_test.append(line.strip())
```

```
# B) Pre-processing: simple cleaning: define regular expressions, apply line by line
```

```
import re
```

```
REPLACE_NO_SPACE = re.compile("[.;:!\'?,\`()\\[\]]")
REPLACE_WITH_SPACE = re.compile("<br\s*/><br\s*/>|(\-)|(\/)")
```

```
def preprocess_reviews(reviews):
    reviews = [REPLACE_NO_SPACE.sub("", line.lower()) for line in reviews]
    reviews = [REPLACE_WITH_SPACE.sub(" ", line) for line in reviews]

    return reviews
```

```
reviews_train_clean = preprocess_reviews(reviews_train)
reviews_test_clean = preprocess_reviews(reviews_test)
```

Your Turn: Use NLTK Text Normalization Tools

```
# C) Tokenization and possibly stemming/lemmatization

import nltk
tokenizer = nltk.tokenize.TreebankWordTokenizer()

def stem_reviews(reviews):
    new_reviews = []
    stemmer = nltk.stem.PorterStemmer()
    for line in reviews:
        tokens = tokenizer.tokenize(line)
        new_reviews.append(" ".join(stemmer.stem(token) for token in tokens))
    return new_reviews

def lemmatize_reviews(reviews):
    new_reviews = []
    lemmatizer = nltk.stem.WordNetLemmatizer()
    for line in reviews:
        tokens = tokenizer.tokenize(line)
        new_reviews.append(" ".join(lemmatizer.lemmatize(token) for token in tokens))
    return new_reviews

reviews_train_stemmed = stem_reviews(reviews_train)
reviews_test_stemmed = stem_reviews(reviews_test)

reviews_train_lemmatized = lemmatize_reviews(reviews_train)
reviews_test_lemmatized = lemmatize_reviews(reviews_test)
```

Edit Distance

How similar are two strings?

... and why do we care?

- Spelling correction:

- **graffe**

- aardvark?
 - woodchuck?
 - giraffe?

- Coreference:

- **Università di Bologna**

- University of Bologna?
 - Aeroporto di Bologna?
 - Università di Pisa?

Minimum Edit Distance

Minimum edit distance: minimum number of **edit operations** needed to transform one string into another:

- **insertion**
- **deletion**
- **substitution**

Example: going from **INTENTION** (source) to **EXECUTION** (target)

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N

This can be seen as an **alignment problem**

Levenshtein Distance

	I	N	T	E	*	N	T	I	O	N
	*	E	X	E	C	U	T	I	O	N
d	s	s				i	s			

Figure 2.14 Representing the minimum edit distance between two strings as an **alignment**. The final row gives the operation list for converting the top string into the bottom string: d for deletion, s for substitution, i for insertion.

A **cost** is assigned to each operation. For instance:

- **insertion** costs 1
- **deletion** costs 1
- **substitution** costs 2 (substituting a letter for itself costs 0)

How Do We Find the Distance?

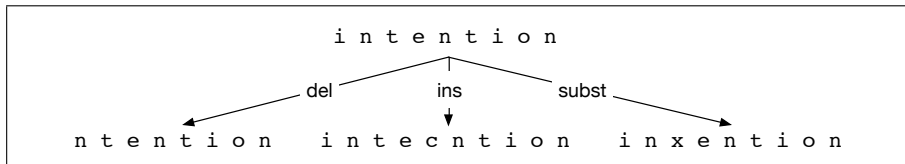


Figure 2.15 Finding the edit distance viewed as a search problem

It's a **search** problem

- Find the **shortest path** from one string to another
- Huge space but many path lead to same destination
- Use **dynamic programming**

Dynamic Programming: Intuition

i n t e n t i o n	← <i>delete i</i>
n t e n t i o n	← <i>substitute n by e</i>
e t e n t i o n	← <i>substitute t by x</i>
e x e n t i o n	← <i>insert u</i>
e x e n u t i o n	← <i>substitute n by c</i>
e x e c u t i o n	

Figure 2.16 Path from *intention* to *execution*.

- If a node x is in the optimal path from A to B , then the optimal path from A to B must include the optimal path from A to x
- Makes sense to memorize and combine solutions to sub-problems

Minimum Edit Distance Algorithm Illustrated

Src\Tar	#	e	x	e	c	u	t	i	o	n
#										
i										
n										
t										
e										
n										
t										
i										
o										
n										

Minimum Edit Distance Algorithm Illustrated

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Figure 2.18 Computation of minimum edit distance between *intention* and *execution* with the algorithm of Fig. 2.17, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

Minimum Edit Distance Algorithm Illustrated

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ 1	↖↖↖ 2	↖↖↖ 3	↖↖↖ 4	↖↖↖ 5	↖↖↖ 6	↖↖↖ 7	↖ 6	← 7	← 8
n	↑ 2	↖↖↖ 3	↖↖↖ 4	↖↖↖ 5	↖↖↖ 6	↖↖↖ 7	↖↖↖ 8	↑ 7	↖↖↖ 8	↖ 7
t	↑ 3	↖↖↖ 4	↖↖↖ 5	↖↖↖ 6	↖↖↖ 7	↖↖↖ 8	↖ 7	↖↖↖ 8	↖↖↖ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖↖ 5	← 6	← 7	↖↖ 8	↖↖↖ 9	↖↖↖ 10	↑ 9
n	↑ 5	↑ 4	↖↖↖ 5	↖↖↖ 6	↖↖↖ 7	↖↖↖ 8	↖↖↖ 9	↖↖↖ 10	↖↖↖ 11	↖↖ 10
t	↑ 6	↑ 5	↖↖↖ 6	↖↖↖ 7	↖↖↖ 8	↖↖↖ 9	↖ 8	← 9	← 10	↖↖ 11
i	↑ 7	↑ 6	↖↖↖ 7	↖↖↖ 8	↖↖↖ 9	↖↖↖ 10	↑ 9	↖ 8	← 9	← 10
o	↑ 8	↑ 7	↖↖↖ 8	↖↖↖ 9	↖↖↖ 10	↖↖↖ 11	↑ 10	↑ 9	↖ 8	← 9
n	↑ 9	↑ 8	↖↖↖ 9	↖↖↖ 10	↖↖↖ 11	↖↖↖ 12	↑ 11	↑ 10	↑ 9	↖ 8

Figure 2.19 When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings. Diagram design after [Gusfield \(1997\)](#).

Spelling Correction

Spelling Correction: a Simple Application of Edit Distance

Two perspectives

- **Non-word spelling correction** (like **graffe** for **giraffe**)
- **Real word spelling correction** deriving from
 - **typographical** errors (insertion, deletion, transposition) that accidentally result in an actual English word, like **there** for **three**
 - **cognitive** errors, like **dessert** for **desert**, or **piece** for **peace**
- Non-word spelling correction can be detected using a **dictionary**
- Then a ranked list of **candidates** can be generated
- Ranking based on **distance metric** such as minimum edit distance
- Preference given to frequent words, or more likely given the context
- In real word spelling error detection, any input word could be an error

The Noisy Channel

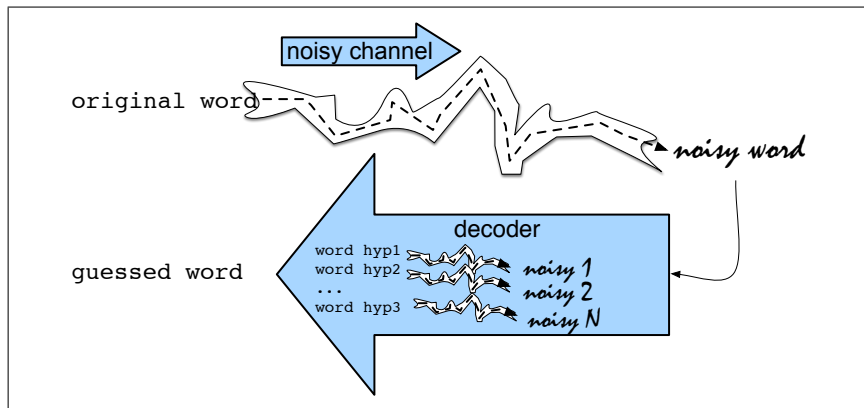


Figure B.1 In the noisy channel model, we imagine that the surface form we see is actually a “distorted” form of an original word passed through a noisy channel. The decoder passes each hypothesis through a model of this channel and picks the word that best matches the surface noisy word.

The Noisy Channel

- The goal is to build a **model of the channel**
- We can then pass every word of the language through the channel, and see which one comes out closest to the observed word
- It's a kind of **Bayesian inference**



- Out of all the words in the vocabulary, find the word **w** such that $P(w|x)$ is highest.

The Noisy Channel

- Our estimate of the correct word is $\hat{w} = \operatorname{argmax}_{w \in V} P(w|x)$



The Noisy Channel

- Our estimate of the correct word is $\hat{w} = \operatorname{argmax}_{w \in V} P(w|x)$

- By Bayes rule:

$$P(w|x) = \frac{P(x|w)P(w)}{P(x)}$$

- Because $\hat{w}$ is independent of $P(x)$, we have

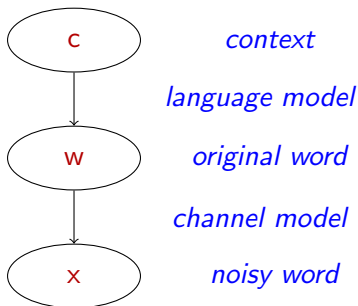
$$\hat{w} = \operatorname{argmax}_{w \in V} P(w|x) = \operatorname{argmax}_{w \in V} P(x|w)P(w)$$

- Finally, we can restrict the analysis to a set of **candidates** $C \subset V$:

$$\hat{w} = \operatorname{argmax}_{w \in C} \underbrace{P(x|w)}_{\text{channel model}} \underbrace{P(w)}_{\text{prior}}$$

What is the Prior Probability of a Word?

- Most likely words in English: the, of, and, a, to, ...



- If we consider the **context**: $\hat{w} = \underset{w \in C}{\operatorname{argmax}} \underbrace{P(x|w)}_{\text{channel model}} \underbrace{P(w|c)}_{\text{language model}}$

Noisy Channel Spelling Method

- ① Find words with similar spelling to input word
 - Majority of errors are single-letter change, including **transpositions**
 - Use **Damerau-Levenshtein** edit distance
 - four edits: *insert, delete, substitute, transpose adjacent*
 - Make list of candidates at distance = 1 edit from input word
- ② Score each candidate based on language model and channel model (aka **error model**)
 - Scoring may take into account typist features
 - e.g., left-handed vs right-handed
 - ... or just look at local context
 - modified letter, misspelling, surrounding letters
 - e.g., to estimate $p(\text{acress}|\text{across})$: how often is e substituted for o?
 - Could use an $|A| \times |A|$ **confusion matrix**

Confusion Matrix

sub[X, Y] = Substitution of X (incorrect) for Y (correct)

X	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	5	0	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

Spell Correction Example: Candidates

*... was called a “stellar and versatile **acress** whose combination of sass and glamour has defined her...”*

Error	Correction	Transformation			
		Correct Letter	Error Letter	Position (Letter #)	Type
acress	actress	t	—	2	deletion
acress	cress	—	a	0	insertion
acress	caress	ca	ac	0	transposition
acress	access	c	r	2	substitution
acress	across	o	e	3	substitution
acress	acres	—	s	5	insertion
acress	acres	—	s	4	insertion

Figure B.3 Candidate corrections for the misspelling *acress* and the transformations that would have produced the error (after [Kernighan et al. \(1990\)](#)). “—” represents a null letter.

MD Kernighan, KW Church, WA Gale, *A spelling correction program base on a noisy channel model*, COLING 1990

Spell Correction Example: Language Model

- Say priors are computed without taking the context into account
- $P(w) = \frac{\text{count}(w)}{|N|}$, where N is the number of tokens in the corpus
- COCA (Corpus of Contemporary English): 404,253,213 words

w	count(w)	p(w)
actress	9,321	.0000231
cress	220	.000000544
caress	686	.00000170
access	37,038	.0000916
across	120,844	.000299
acres	12,874	.0000318

Spell Correction Example: Error Model

Candidate Correction	Correct Letter	Error Letter	$x w$	$P(x w)$
actress	t	-	c ct	.000117
cress	-	a	a #	.00000144
caress	ca	ac	ac ca	.00000164
access	c	r	r c	.000000209
across	o	e	e o	.0000093
acres	-	s	es e	.0000321
acres	-	s	ss s	.0000342

Figure B.4 Channel model for *acress*; the probabilities are taken from the *del[]*, *ins[]*, *sub[]*, and *trans[]* confusion matrices as shown in [Kernighan et al. \(1990\)](#).

Spell Correction Example: Putting It All Together

Candidate	Correct	Error				
Correction	Letter	Letter	$x w$	$P(x w)$	$P(w)$	$10^9 * P(x w)P(w)$
actress	t	-	c ct	.000117	.0000231	2.7
cress	-	a	a #	.00000144	.000000544	0.00078
caress	ca	ac	ac ca	.00000164	.00000170	0.0028
access	c	r	r c	.000000209	.0000916	0.019
across	o	e	e o	.0000093	.000299	2.8
acres	-	s	es e	.0000321	.0000318	1.0
acres	-	s	ss s	.0000342	.0000318	1.0

Figure B.5 Computation of the ranking for each candidate correction, using the language model shown earlier and the error model from Fig. B.4. The final score is multiplied by 10^9 for readability.

- Almost.. **actress** is only second most likely word
- Why?

A Better Language Model?

*... was called a “stellar and versatile **acress** whose combination of sass and glamour has defined her...”*

- Say $P(w_i) = P(w_i|w_{i-1})$, and
 - $P(\text{actress}|\text{versatile}) = .000021$
 - $P(\text{across}|\text{versatile}) = .000021$
 - $P(\text{whose}|\text{actress}) = .001$
 - $P(\text{whose}|\text{across}) = .000006$
- Then:
 - $P(\text{versatile actress whose}) = .000021 \times .001 = 210 \times 10^{-10}$
 - $P(\text{versatile across whose}) = .000021 \times .000006 = 1 \times 10^{-10}$
- Now:
 - $P(\text{versatile actress whose}|\text{versatile across whose}) = 2.7 \times 210 \times 10^{-19}$
 - $P(\text{versatile across whose}|\text{versatile across whose}) = 2.8 \times 10^{-19}$

Questions?